Information Processing 2

introduction to C programming

end of semester projects (weeks 13–15)

You can earn up to 50 points towards your final grade from projects. Choose, complete, and present any combination of projects that you like. Ask the instructors or TAs if you need help.

Your project work **must** be graded **in-class** by an instructor. We do not accept project submissions electronically.

Each project can be completed as a single '`.c`' source file or (if it makes sense) as two or more source files with any header files that are needed for them to work properly together.

When grading your project we will ask you questions about how it works. You will earn some of the available points for demonstrating a working project and the rest for your ability to explain its operation.

Contents

1 Prime number generator [10 points]

Write a prime number generator using the *Sieve of Eratosthenes*.[1] Your program should require exactly one command-line argument, an integer (no smaller than 2) specifying the largest prime number to print. An example session might be as follows:

```
$ ./primes
usage: ./primes max-prime
$ ./primes 0
./primes: max-prime 0 is too small (must be at least 2)
$ ./primes 20
2 3 5 7 11 13 17 19
```

To find the prime numbers up to some maximum n, an array $p[i]$ with indices $2 \leq i \leq n$ is used. Each element $p[i]$ is *true* if i is prime and *false* if it is not. Initially every element in p is set to *true*. For each i between 2 and n, the element $p[i]$ is tested. If $p[i]$ is *false* then the number is not prime and can be ignored. If $p[i]$ is *true* then then i is prime and all multiples of it $p[n \times i]$ (for $n \geq 1$) in the array can be set to *false*, since they are multiples of a prime factor i. When the process completes, the elements $p[i]$ that are *true* identify prime numbers i. As pseudo-code:

```
1 let P be an array of boolean values indexed by integers 2 to N
2 set all elements of P to true
3 for i in 2 ... N do
4   if P[i] is true then
5     for j in 2i ... N do
6       set P[j] to false
7 for i in 2 ... N do
8   if P[i] is true then
9     print i
```

Note that two worthwhile optimisations are possible. First, when a new prime i is discovered (at line 4) all composite (non-prime) numbers with factors $< i$ have already been removed from p. The loop on line 5 can therefore begin removing multiples of i starting with i^2 (rather than $2i$). Second, since every composite number n must have at least one factor $f \leq \sqrt{n}$, it is sufficient that the main loop on line 3 step i from 2 to $\sqrt{N}$ to guarantee that every non-prime number is removed from P.

Test your program by generating and printing the prime numbers between 2 and 100 (inclusive).

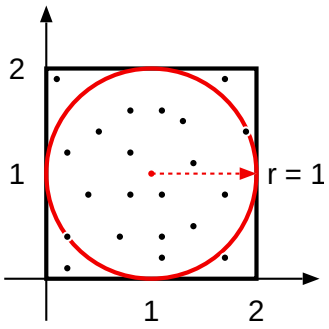
1.1 Deliverable

Present the entire program which we will test with various command-line arguments.

[1] https://en.wikipedia.org/wiki/Sieve_of_Eratosthenes

2 Monte Carlo simulation to calculate π [10 points]

Write a program to approximate π. You can do this by generating random points inside a 2×2 square and then counting what fraction of the points land inside a unit circle that is concentric with the square. The test for landing inside the circle is easy: the distance between the point and the centre of the square, calculated using Pythagoras' Theorem, must be < 1.



A graphical illustration of the simulation is shown on the right. Random points in the square bounded by $(0,0)$ and $(2,2)$ are generated. The number of points lying within the circle are tallied. The 2×2 square has an area of 4. The circle has an area of $\pi r^2 = \pi$ (since $r = 1$). The number of points lying inside the circle should therefore be $\frac{\pi}{4}$ of the total number of points. If the total number of points is (for example) $4,000,000$ then the number lying inside the circle should be approximately $1,000,000 \times \pi$.

The <stdlib.h> function `int rand(void)` returns random integers in the range $[0 \ldots$ `RAND_MAX`$]$. Convert these to random doubles in the range $[0 \ldots 1]$ and then multiply them by 2.0 to make x and y coordinates of random points within the square. For each point use Pythagoras' Theorem to calculate its distance d from $(1,1)$. If $d < 1$ then the point is inside the circle. Use the ratio of points inside the circle to the total number of points to calculate and print π.

Your program should accept one command-line argument, the total number of points to generate and test. Using `int` counters you can achieve no more than 9 significant digits of accuracy (why?) and so printing 8 digits after the decimal point is appropriate. An example session might be as follows:

```
$ ./pi
usage: ./pi n-points
$ ./pi 4000000
3142106 out of 4000000 points inside the circle
pi is approximately 3.14210600
```

2.1 Deliverable

Present the entire program which we will test with various command-line arguments.

3 Improved RPN calculator [5 points]

The slides for Week 8 show a reverse-Polish notation calculator. An example session, that calculates $1+2\times3+4$, might be as follows:

```
$ ./calc
1 2 3 * 4 + +
        11
```

Modify the calculator so that it uses `scanf()` to read numbers and operators. This should allow you to omit the space after numbers in your expressions, but if you find that you still need to read one character too much from the input then also use `ungetc()` to push the 'one character too much' back onto the standard input.

3.1 Deliverable

Present the entire program which we will test with various computations.

4 Add variables to a RPN calculator [10 points]

Prerequisite: an updated RPN calculator using `scanf` (and possibly `ungetc`) (see Project 3) will make a better starting point for this project than the version shown in the slides for Week 8.

Project 3 (and the Week 8 slides) show a reverse-Polish notation calculator. An example session, that calculates $1+2\times3+4$, might be as follows:

```
$ ./calc
1 2 3 * 4 + +
        11
```

Modify the calculator so that it supports 26 single-letter variables `a` through `z`. (Variable names should ignore case: `a` and `A` should both refer to the variable `a`. Hint: the `<ctype.h>` library function `int tolower(int c)` can help you achieve this.) A typical session might now look like this:

```
$ ./calc
a = 1 2 3 * 4 + +
        11
b = a a +
        22
20 b +
        42
```

Fewer than 20 additional lines of code are needed to implement variables, provided you make some simplifying assumptions:

- only one assignment is allowed per input line;
- no matter where the variable name and = symbol appear on a line, the assignment is always performed at the end of the line when a newline character is encountered; and
- `getop(char s[])` could handle a variable by printing its value as text into the array `s` used to communicate a `NUMBER` back to the main program, even though this might introduce some small rounding errors.

4.1 Deliverable

Present the entire program which we will test with various computations.

5 A 3×3 matrix data type [5 points]

Create a 3×3 matrix type that supports initialisation from constants, printing, and matrix multiplication.

5.1 Data type

Define a type `Mat3` that holds a 3×3 matrix of `double` values. Make sure you can initialise your matrices using constants, like this.

```
Mat3 mat3 = { { 1, 2, 3 }, { 4, 5, 6 }, { 7, 8, 9 } };
```

5.2 Printing

Write a function `void Mat3_print(Mat3 M)` that prints the contents of the matrix `M`. At this point you should be able to run this program.

```
int main()
{
  Mat3 mat3 = { { 1, 2, 3 }, { 4, 5, 6 }, { 7, 8, 9 } };
  Mat3_print(mat3);
  return 0;
}
```

The output could look like an initialiser for a `Mat3`, for example:

```
{ { 1.000000, 2.000000, 3.000000 },
  { 4.000000, 5.000000, 6.000000 },
  { 7.000000, 8.000000, 9.000000 } }
```

5.3 Arithmetic

Write a function `void Mat3_mulMat3(Mat3 A, Mat3 B, Mat3 P)` that multiplies the matrix `A` by the matrix `B`, placing the result in matrix `P`. This program

```
  Mat3 A = { { 2, 1, 5 }, { 2, 10, 5 }, { 3, 1, 4 } };
  Mat3 B = { { 8, 7, 1 }, { 4,  2, 7 }, { 2, 3, 5 } };
  Mat3 P;
  Mat3_mulMat3(A, B, P);
  Mat3_print(P);
```

should print the following result:

```
{ { 30.000000 31.000000 34.000000 }
  { 66.000000 49.000000 97.000000 }
  { 36.000000 35.000000 30.000000 } }
```

5.4 Deliverable

Present the program (using the declarations, computations, and printing described above) which we will verify.

6 3-element vector data type [10 points]

Prerequisite: a working `Mat3` data type (see Project 5).

Create a 3-element vector data type that supports initialisation from constants, printing, and multiplication by a 3×3 matrix. The function `void Mat3_mulVec3(Mat3 M, Vect3 V, Vec3 P)` multiplies the matrix `M` by the vector `V` and stores the result in the vector `P`. You can check your implementation with the following program. (The expected output is shown after each print operation.)

```
  Vec3 V = { 3, 4, 1 };
  Vec3_print(V);

{ 3.000000, 4.000000, 1.000000 }

  Mat3 S = { { 3, 0, 0 }, {  0, 2, 0 }, { 0, 0, 1 } };
  Vec3 SV;
  Mat3_mulVec3(S, V, SV);
  Vec3_print(SV);

{ 9.000000, 8.000000, 1.000000 }

  Mat3 T = { { 1, 0, 5 }, {  0, 1, 7 }, { 0, 0, 1 } };
  Vec3 TV;
  Mat3_mulVec3(T, V, TV);
  Vec3_print(TV);

{ 8.000000, 11.000000, 1.000000 }

  Mat3 TS;
  Mat3_mulMat3(T, S, TS);

  Vec3 TSV;
  Mat3_mulVec3(TS, V, TSV);
  Vec3_print(TSV);

{ 14.000000, 15.000000, 1.000000 }
```

6.1 Deliverable

Present your completed program (using the declarations, computations, and printing described above) which we will verify.

6.2 Just for fun

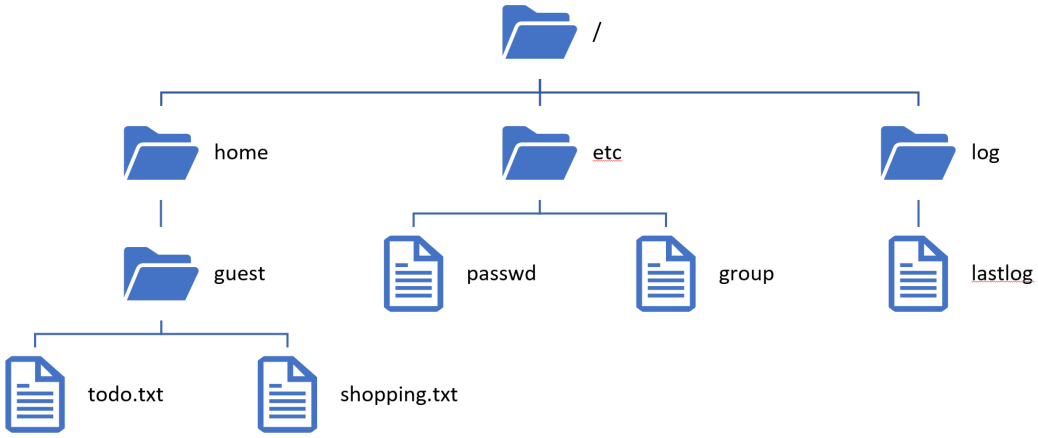
If you consider the vector `V` to represent the (2-dimensional) point $(3,4)$ then, based on the results of the multiplications, can you guess what the matrices `S` and `T` might represent? If you can then test your theory by modifying the vector and matrices to see if the results are what you expect.

Can you now guess what the matrix product $T \times S = TS$ represents when applied to a vector `V`? If you have a theory about this then test it by modifying `S` and `T` and comparing the product $TS \times V$ to your predicted result.

7 Printing the file and directory names in a hierarchy [10 points]

Write a program that 'walks' a directory hierarchy, from a specified starting point, and prints the names of the files and directories that it encounters.

Your program should accept zero or one command-line argument. With one argument, it specifies the path to the hierarchy to be printed. With no argument, it should start in the current working directory '.'. Your program should subsequently ignore the two standard directory entries '.' and '..' in any sub-directories that it traverses. Given the following hierarchy



the output of the program might look like this:

```
$ ./listfiles /home
/home
/home/guest
/home/guest/todo.txt
/home/guest/shopping.txt
$ ./listfiles /
/
/etc
/etc/passwd
/etc/group
/home
/home/guest
/home/guest/todo.txt
/home/guest/shopping.txt
/log
/log/lastlog
```

The easiest way to write the program is as a recursive function that prints the name of a file or a directory, and in the case of a directory will then open it and recursively call itself to print each entry in the directory. Note that when the recursive function finds a sub-directory it will have to explicitly concatenate the name of the parent directory, a '/', and the name of the sub-directory to create the path name that it needs to recursively call itself. Note also that on Windows (using mingw) the `dirent` structure does not contain a `d_type` field that would allow you to test for a regular file or a directory. Instead you will have to use the `stat(pathname, &statbuf)` function on each path name and then test the `statbuf.st_mode & S_IFDIR` bit to determine whether the path name refers to a normal file or to a directory.

7.1 Deliverable

Present the completed program which we will test with various different command-line arguments.

8 Searching for specific names in a hierarchy [5 points]

Prerequisite: a working recursive file listing program (see Project 7).

Add a second command-line argument to your file listing program. If the second command-line argument is present then it is a search string. The program should only print the filenames that include the search string somewhere in the name. Make sure that your program handles search strings that contain a '/' character.

Given the hierarchy shown in Project 7, the output of the program might look like this:

```
$ ./listfiles /home txt
/home/guest/todo.txt
/home/guest/shopping.txt
$ ./listfiles / s
/etc/passwd
/home/guest
/home/guest/todo.txt
/home/guest/shopping.txt
/log/lastlog
```

8.1 Deliverable

Present the completed program which we will test with various different command-line arguments.

A Installing and using the SDL2 graphics libraries

To support both graphics and text you need two libraries and their associated header files.

- `libSDL2` provides a window and some simple drawing functions.
- `libSDL2_ttf` adds support for drawing of text into a SDL2 window.

Installation is different depending on which platform and compiler you are using. See the relevant section below.

Once the software is installed and tested you can use the file `template.c` as a template for your own graphics programs. Copy it to a file called `myprogram.c`, modify it as you like, and then build your program using the `make` command. That will run the compiler with the correct options for your platform. For example, on Linux:

```
$ make myprogram
gcc -Wall -g template.c  -lSDL2_ttf -lSDL2 -lm -o template
$ ./myprogram
```

A.1 Linux

First, install the SDL2 libraries using the package manager. On Debian or Ubuntu the command is:

```
$ sudo apt install libsdl2-dev libsdl2-ttf-dev
```

Second, download the archive `window-SDL-linux.tgz` (from the course web page or Teams) and then unpack it using the `tar` command.

```
$ tar xvfj window-SDL-linux.tgz
```

This will create a directory called `linux` in your current directory. Compile and run the test programs:

```
$ cd linux
$ make
$ ./example1
$ ./example2
```

A.2 Mac using MacPorts

First, install the SDL2 libraries.

```
$ sudo port selfupdate
$ sudo port install libsdl2 libsdl2_ttf
```

Second, download the archive `window-SDL-macports.tgz` (from the course web page or Teams) and then unpack it using the `tar` command.

```
$ tar xvfj window-SDL-macports.tgz
```

This will create a directory called `macports` in your current directory. Compile and run the test programs:

```
$ cd macports
$ make
$ ./example1
$ ./example2
```

A.3 Mac without MacPorts

First, install the SDL2 libraries.

???

Second, download the archive `window-SDL-macos.tgz` (from the course web page or Teams) and then unpack it using the `tar` command.

```
$ tar xvfj window-SDL-macos.tgz
```

This will create a directory called `macos` in your current directory. Compile and run the test programs:

```
$ cd macos
$ make
$ ./example1
$ ./example2
```

A.4 Windows using Windows Subsystem for Linux

Running graphical programs under WSL is tricky. The easiest way is to install a *cross compiler* that generates Windows binaries and then compile native Windows graphical applications from within WSL.

First, install the cross compiler using the package manager:

```
$ sudo apt install gcc-mingw-w64-x86-64
```

Second, download the archive `window-SDL-wsl.tgz` (from the course web page or Teams) and then unpack it using the `tar` command.

```
$ tar xvfj window-SDL-wsl.tgz
```

This will create a directory called `wsl` in your current directory. Compile and run the test programs:

```
$ cd wsl
$ make
$ ./example1
$ ./example2
```

A.5 Windows using MinGW

Download the archive `window-SDL-mingw.tgz` (from the course web page or Teams) and then unpack it using the `tar` command.

```
$ tar xvfj window-SDL-mingw.tgz
```

This will create a directory called `mingw` in your current directory. Compile and run the test programs:

```
$ cd mingw
$ make
$ ./example1
$ ./example2
```

B Using the graphics library

At the start of your program, add

```
#include "window.c"
```

to include the library source in your program or

```
#include "window.h"
```

if you will compile `window.c` separately. If you are not using a `Makefile` then you will have to add options to your compile command:

`gcc -I` *path-to-header-files* `-L` *path-to-library-files* `-l SDL2 -l SDL2_ttf -lm`

On Windows you maybe also have to add one or more of the following to the compile command:

```
-l mingw
-l SDL2main
```

B.1 Creating a window

Before performing any other operation, your program should call

```
size(int width, int height);
```

which will create a window of the given *width* and *height*, measured in pixels. The same function can be called again at any time to resize the window. The global integer variables `width` and `height` contain the current width and height of the window.

The function

```
title(char *name);
```

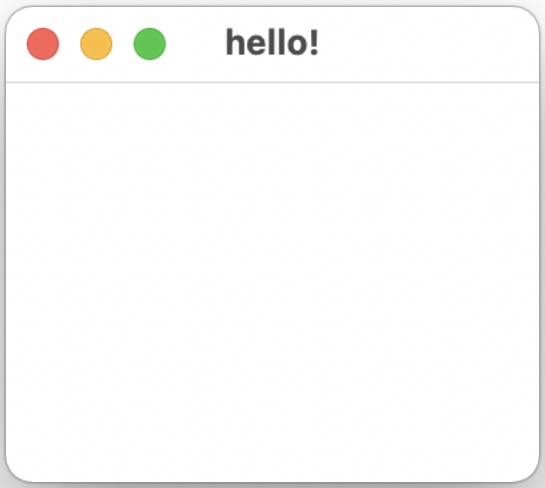
sets the name of the window displayed in the title bar. When your program is not drawing or processing input events, it can call

```
delay(int milliseconds)
```

to pause execution for the given number of *milliseconds*. While paused the program will still react to input events normally.

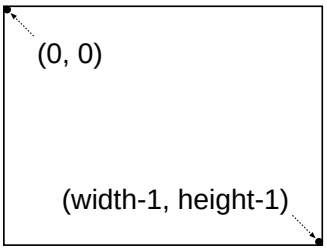
```
#include "window.c"

int main()
{
  size(200, 150);
  clear();
  title("hello!");
  delay(10000);
  return 0;
}
```



B.2 Window coordinates

The origin is in the top left corner of the window. X coordinates begin on the left with column `0` and increase rightwards to the rightmost column `width-1`. Y coordinates begin on the top line with row `0` and increase downwards to the bottom row `height-1`.



B.3 Colours

Drawing occurs using three colours. The *background* colour is used to fill the empty window each time it is cleared. The *stoke* colour is used to draw points, lines, and the outlines of solid figures (rectangles and polygons). The *fill* colour is used to fill the interiors of solid figures (rectangles and polygons). All three colours can be changed.

`void background(Colour colour)`
: sets the *background* colour that will be used to clear the window.

`void fill (Colour colour)`
: sets the *fill* colour that will be used to fill the interiors of solid figures.

`void stroke (Colour colour)`
: sets the *stroke* colour that will be used to draw points, lines, and the outlines of solid figures.

Values of type `Colour` describe colours. They are created by combining different intensities of red, green, and blue components. The following macros and functions create `Colour` values.

`Colour RGB(int red, int green, int blue)`
: creates a `Colour` as a blend of three primary colour components *red*, *green*, and *blue*; the intensity of each component must be in the range `0` (fully off) to `255` (fully on).

`Colour GREY(P)`
: creates a shade of grey as a percentage of white; `P` must be in the range `0` (black) to `100` (white).

For convenience, the following named colours are defined as constants:

Levels of grey	`BLACK`, `GREY25`, `GREY50`, `GREY75`, `WHITE`
Primary colours	`RED`, `GREEN`, `BLUE`
Secondary colours	`CYAN`, `MAGENTA`, `YELLOW`
No colour (invisible)	`TRANSPARENT`

Drawing with the colour `TRANSPARENT` has no effect on the content of the window.

```c
#include "window.c"

int main()
{
  size(200, 150);
  background(RED);
  clear();
  delay(10000);
  return 0;
}
```

B.4 Advanced topics

`Colour`s can be created using the HSV model and they can also have transparency.

B.4.1 HSV colours

An alternative to the RGB colour model is hue-saturation-value or HSV. The HSV model is based on human visual perception and can be a more intuitive way to select colours.

The *hue* is a colour angle in the range `0` to `359`. Red is at `0`, yellow at `60`, green at `120`, cyan at `180`, blue at `240`, and magenta at `300` degrees. Intermediate angles blend the adjacent colours proportionally.

The *saturation* describes the amount of grey in the colour as a percentage in the range `0` to `100`. Lowering saturation towards `0` increases the amount of grey, producing a 'faded' colour. Raising saturation towards `100` decreases the amount of grey, producing more 'vibrant' colours.

The *value* describes the brightness or intensity of the colour as a percentage in the range `0` to `100`. Lowering the value to `0` gives black and raising it to `100` gives the brightest colour or shade of grey.

`Colour HSV(double hue, double saturation, double value)`
creates a `Colour` using the hue-saturation-value model.

B.4.2 Transparency

Partially transparent colours can also be created by specifying an *apha* value in addition to the three primary colour intensities.

`Colour RGBA(int red, int green, int blue, int alpha)`
creates a `Colour` as a blend of three primary colour components *red*, *green*, and *blue* with opacity *alpha*. The intensity of each component must be in the range `0` (fully off) to `255` (fully on). The opacity *alpha* must be in the range `0` (fully transparent) to `255` (fully opaque).

When replacing one colour with another during drawing, the *alpha* value of the new colour controls how it is blended with the old colour that already exists at the same location. If

$$\alpha = \texttt{alpha} \div 255.0$$

then the final colour will be:

$$\alpha \times newColour + (1 - \alpha) \times oldColour$$

An alpha of `0` therefore leaves the old colour unchanged, an alpha of `128` blends the new colour equally with the old colour, and an alpha of `255` completely replaces the old colour with the new colour.

If you use a background colour that is partially transparent then the window will not completely erase its contents when you call `clear()`. This can be used to create a kind of 'visual persistence' effect.

`unsigned char )`
sets the *background* colour that will be used to clear the window.

`void fill (Colour colour)`
sets the *fill* colour that will be used to fill the interiors of solid figures.

`void stroke (Colour colour)`
sets the *stroke* colour that will be used to draw points, lines, and the outlines of solid figures.

```
typedef union Colour Colour;
```

B.5 Drawing

Before drawing its content, the window can be cleared to the current background colour by calling:

```
clear();
```

Points, lines, rectangles, and polygons are drawn with the following functions.

`void drawPoint(int x, int y)`
draws a single point at position (x, y) using the current stroke colour..

`void drawLine(int x1, int y1, int x2, int y2)`
draws a line from position $(x1, y1)$ to position $(x2, y2)$ using the current stroke colour.

`void fillRectangle(int l, int t, int r, int b)`
draws a solid rectangle whose top-left corner is (l, t) and whose bottom-right corner is (r, b) using the current stroke colour.

`void drawRectangle(int l, int t, int r, int b)`
draws a hollow rectangle whose top-left corner is (l, t) and whose bottom-right corner is (r, b) using the current stroke colour. If the current fill colour is not `TRANSPARENT` then the interior of the rectangle is filled using that colour.

`void fillPolygon(int nPoints, int *points)`
draws a solid polygon described by the array `points` using the current stroke colour. The array `points` contains $2 \times$ `nPoints` integers. Each pair of integers in the array `points` describes one vertex of the polygon. The vertices are therefore stored as $(\texttt{points}[2i], \texttt{points}[2i+1])$ for i in $0 \ldots \texttt{nPoints} - 1$. The interior of the polygon is determined using the "even-odd" rule; if any of the the edges intersect then parts of the interior of the polygon will be hollow.

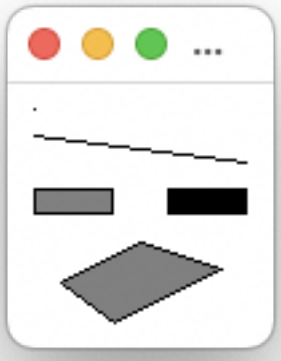
`void drawPolygon(int nPoints, int *points)`
draws a hollow polygon described by the array `points` using the current stroke colour. If the current fill colour is not `TRANSPARENT` then the interior of the polygon is filled using that colour (similarly to `fillPolygon`).

```
#include "window.c"

int main()
{
    size(100, 100);
    clear();
    drawPoint(10, 10);
    drawLine(10,20, 90,30);
    drawRectangle(10,40, 40,50);
    fillRectangle(60,40, 90,50);
    int points[] = {
      50,60, 80,70, 60,80, 40,90, 20,75
    };
    drawPolygon(5, points);
    delay(10000);
    return 0;
}

void font(const char *path, int size);
void drawText(int x, int y, char *text);

void update(void);
```



```
void onKeyDown  (void (*handler)(int));
void onKeyUp    (void (*handler)(int));
void onTextInput(void (*handler)(char *));
void onMouseMove(void (*handler)(int, int));
void onMouseDown(void (*handler)(int, int));
void onMouseUp  (void (*handler)(int, int));

void nullHandler();

void delay(int milliseconds);

unsigned long millisecondClock(void);

#endif // __window_h
```